Multi-Class Machine Learning Approaches to Student Dropout

A Comparative Study of Classifiers and Data Balancing Techniques

AUTHOR
Kareem D. Piper (Advisor: Dr.Shusen Pu)

PUBLISHED
February 17, 2026

Slides: slides.html (Go to `slides.qmd` to edit)

> **Important**
>
> **Placeholder**

1 Introduction

The goal of this capstone project is to test, assess and report the results various multi-class machine learning prediction models on their ability to accurately predict student dropout in the higher education setting. According to Realinho et al. (2022) student attrition and academic failure not only negatively affect the education institutions that students attend, but create greater societal issues. For example, when students dropout prior to completing a degree program it makes them less competitive concerning the job market, which then leads to economic deficiencies, which can lead to inadequate health care access and subsequent more dire socio-economic issues that often disproportionately affect marginalized demographics (Realinho et al., 2022).

According to Aina et al. (2022) statistics released by the Organization of Economic Co-Operation and Development (OECD) state that though the proportion of students enrolling and graduating colleges and universities far exceeds those dropping out, dropout still represents a third of said students. Further, Aina et al. (2022) assert that thirty percent of students in united States dropout, with many of them being early dropouts (i.e., first year of college). Student dropout is a complex issues and there are many factors that attribute to it (Aina et al., 2022). The factors that attribute to student dropout boarder sociological, economical and psychological domains as such according to Aina et al. (2022) scholars approach the phenomena from their respective fields of expertise. There is a need for a more holistic lens when researching the causes of student dropout to better understand the relationships between the factors that attribute to it thus gaining insights on how best to mitigate it through intervention (Aina et al., 2022).

Machine Learning provides the ability for researchers to approach student dropout in a holistic manner specifically because models can converge high dimensional data sets (Raschka et al., 2022). Researchers (e.g., Realinho et al., 2022; Ridwan et al., 2024) have utilized machine learning methods to predict student dropout. However, according to Mduma (2023), the prediction of student dropout is a particularly challenging issues for education researchers, this is due in large part to class imbalance. According to Mduma (2023) many real world datasets concerning student dropout are largely skewed towards the retained or enrolled classes. Many scholars pay particular attention to feature engineering methods when using machine learning methodologies to address the issue of predicting student

attrition. However, Mduma (2023) asserted that class imbalance presents limitations concerning model accuracy and generalization and thus framed their study around that issue.

This capstone project, focuses on the appropriate data preprocessing methods, feature engineering methods, and methods to adequately address class imbalance. Further, I specifically use machine learning models that have a proven track record in the literature (e.g., Realinho et al., 2022; Ridwan et al., 2024) concerning their ability to address multi-class datasets, for example Random Forest(RF), Extreme Gradient Boosting (XGBoost), Light Gradient Boosting Machine (LightGBM) and Categorical Boosting (CatBoost).

2 Research Questions

The purpose of this capstone project is to assess and report the results various multi-class machine learning prediction models on their ability to accurately predict student dropout in the higher education setting using the (UCI Student Dropout Dataset). To inform this capstone project's purpose I asked the following primary and sub-research questions.

Primary Research Question

RQ1: Which multi-class machine learning classifier achieves the highest predictive performance as measured by; accuracy, precision, and F1-score when applied to the (UCI Student Dropout Dataset)?

RQ1a: How do methods of addressing class imbalance affect the predictive performance of multi-class machine learning classfiers?

RQ1b: How does the performance of gradient boosting models compare to tradtional ensable models in multi-class student attrition prediciton?

3 Methods

The main research question of this capstone project centers around comparing multi-class machine learning classifiers and addressing class imbalance, to accurately predict student dropout in a higher education setting. Thus, the methods I use correlate directly to those used by (e.g., Mduma, 2023; Realinho et al., 2022). For example, like the dataset used by Realinho et al. (2022), the target in this capstone's dataset has three categories (i.e, dropout, enrolled, and graduate), which requires the use of multi-class classifiers. As is the case with many real world datasets the classes in the (UCI Student DropoutDataset) are skewed towards enrolled and graduated. To address the issue of class imbalance Mduma (2023), I use five varied data balancing techniques each paired with multi-class machine learning classifier to determine which pairing most accurately predicted student dropout. The subsequent sections of these methods are broken down into to parts, 1. Multi-Class Methods and 2. Data Balancing Methods, where I provide a detailed account of how both are incorporated in this capstone's machine learning pipeline.

As mentioned previously the dataset I am using stems from the UC Irvine Machine Learning Repository and can be accessed here:(UCI Student Dropout Dataset) (Realinho et al., 2022). Realinho et al. (2022)

characterize the dataset as tubular, and state that it is meant for the social sciences, specifically for researchers working on classification based projects. There are (n = 4,424) cases and (n = 36) features in the dataset. The feature types according to Realinho et al. (2022) consist of real, categorical and integers based features. According to Realinho et al. (2022) the dataset was created specifically to help researchers interested in studying the factors that contribute to student dropout or academic failure in higher education using machine learning methods. There target has three categories (i.e., dropout, enrolled and graduate). According to Realinho et al. (2022) the dataset was rigorously preprocessed and consist of no null or missing values.

To better understand the dataset I conducted exploratory data analysis (EDA) on the (UCI Student Dropout Dataset). My goal was to gain a deeper understanding of the datasets' features, their relationship to eachother and the target classes. EDA is a fundamental step in the machine learning pipeline as data may present limitations concerning the chosen methods (Raschka et al., 2022).

▶ Code

▶ Code

```
Repo root: /Users/kareempiper/Projects/IDC6940_DataPraxisAI
Train exists: True /Users/kareempiper/Projects/IDC6940_DataPraxisAI/data/train.csv
Test exists: True /Users/kareempiper/Projects/IDC6940_DataPraxisAI/data/test.csv
Train shape: (3539, 37)
Test shape: (885, 37)
```

▶ Code

▶ Code

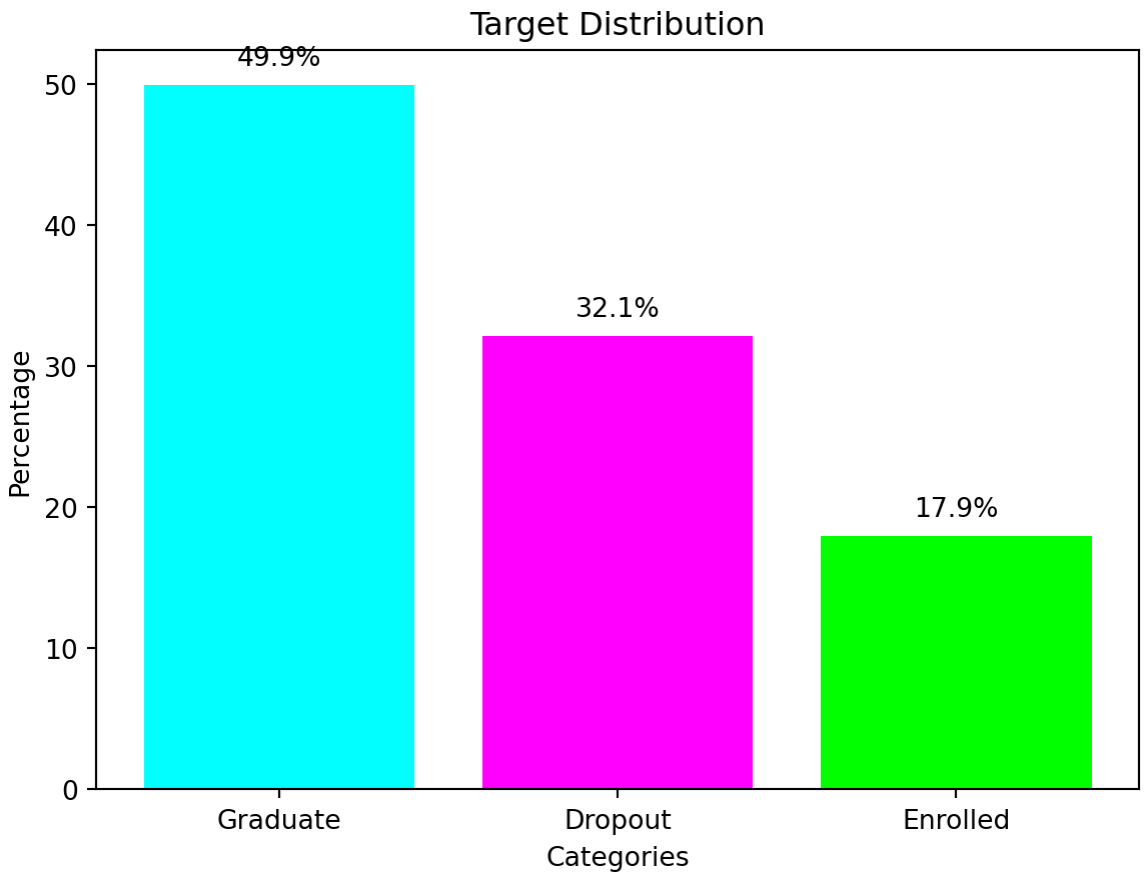
3.0.1 Table 1: Descriptive Statistics for Selected Numerical Features

Statistic	Age at Enrollment	Admission Grade	Curricular Units 2nd Sem (Grade)	Unemployment Rate	Inflation Rate	GDP
Count	3539.0	3539.0	3539.0	3539.0	3539.0	3539.0
Mean	23.3	126.88	11.23	11.5	1.23	1.76
Std	6.13	14.53	2.94	2.62	1.63	2.33
Min	17.0	95.0	0.0	7.6	-0.8	-4.06
25%	19.0	117.8	10.33	9.4	0.3	0.58
50%	21.0	126.0	12.0	11.1	1.4	1.74
75%	25.0	134.9	13.0	13.9	2.6	3.26
Max	70.0	190.0	18.0	16.2	3.7	3.51

▶ Code

▶ Code

▶ Code



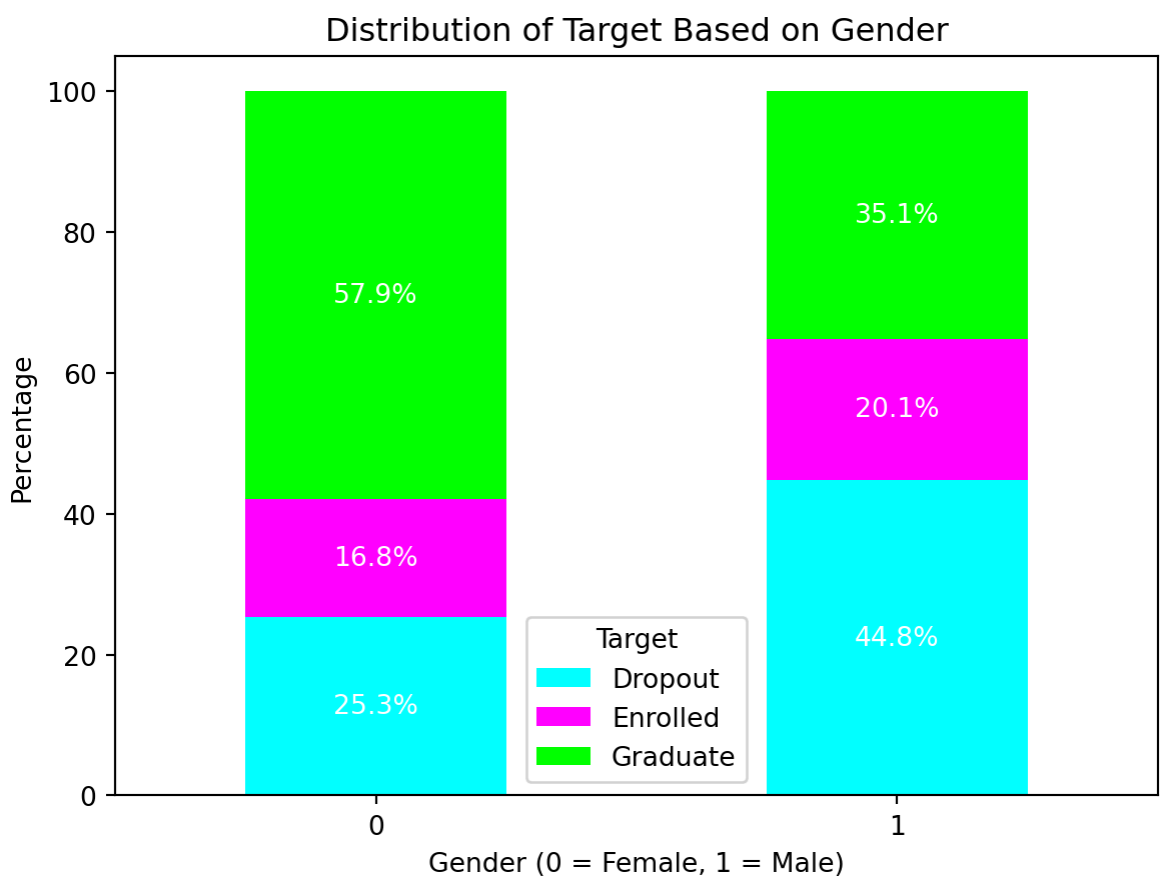
▶ Code

3.0.2 Table 2: Unique Value Counts per Feature (Top and Bottom)

Feature	Unique Count
Daytime/evening attendance	2
Displaced	2
Debtor	2
Educational special needs	2
International	2
Scholarship holder	2
Gender	2

Feature	Unique Count
Tuition fees up to date	2
Marital status	6
Application order	8
...	...
Age at enrollment	46
Previous qualification (grade)	99
Admission grade	591
Curricular units 2nd sem (grade)	674
Curricular units 1st sem (grade)	683

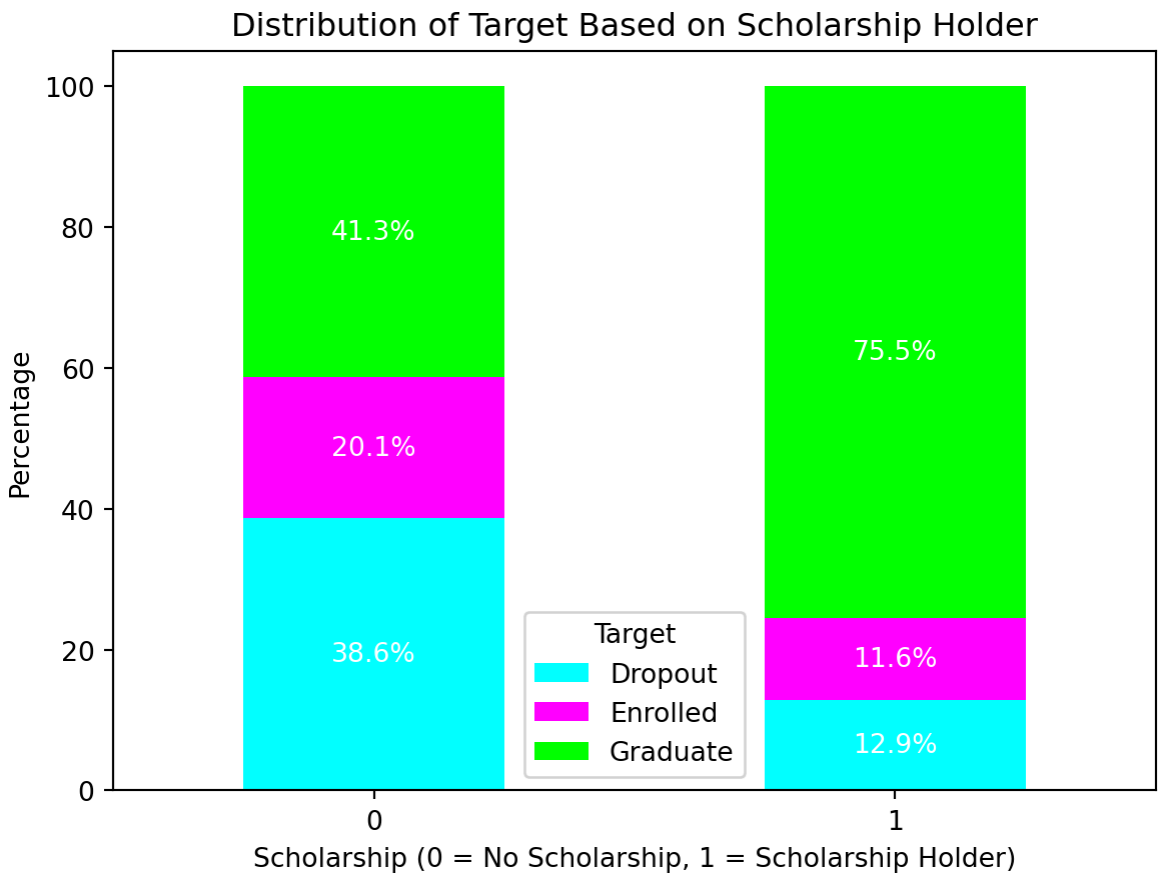
▶ Code



3.0.3 Table 3: Target Distribution by Gender (Percentages)

Gender	Dropout	Enrolled	Graduate
0	25.28	16.80	57.92
1	44.80	20.06	35.13

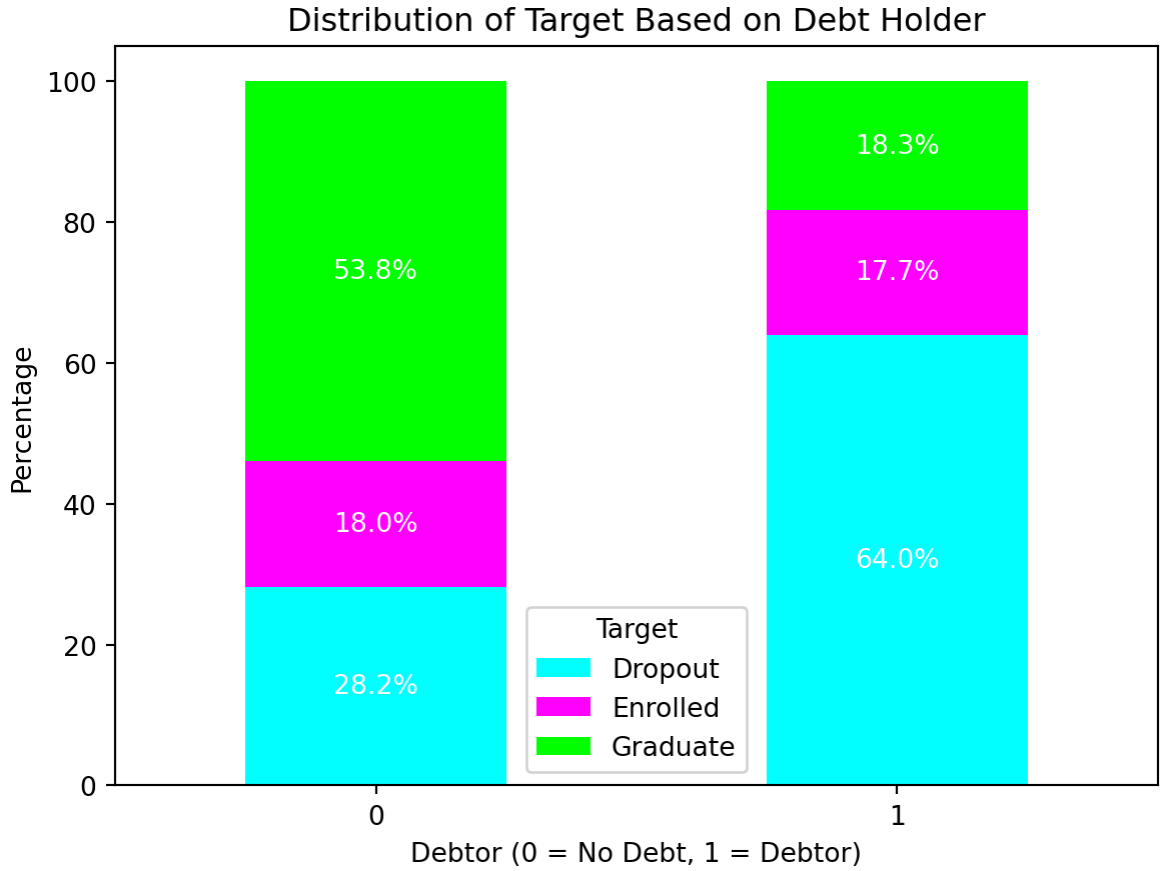
▶ Code



3.0.4 Table 4: Target Distribution by Scholarship Holder Status (Percentages)

Scholarship Holder	Dropout	Enrolled	Graduate
0	38.64	20.08	41.29
1	12.86	11.63	75.50

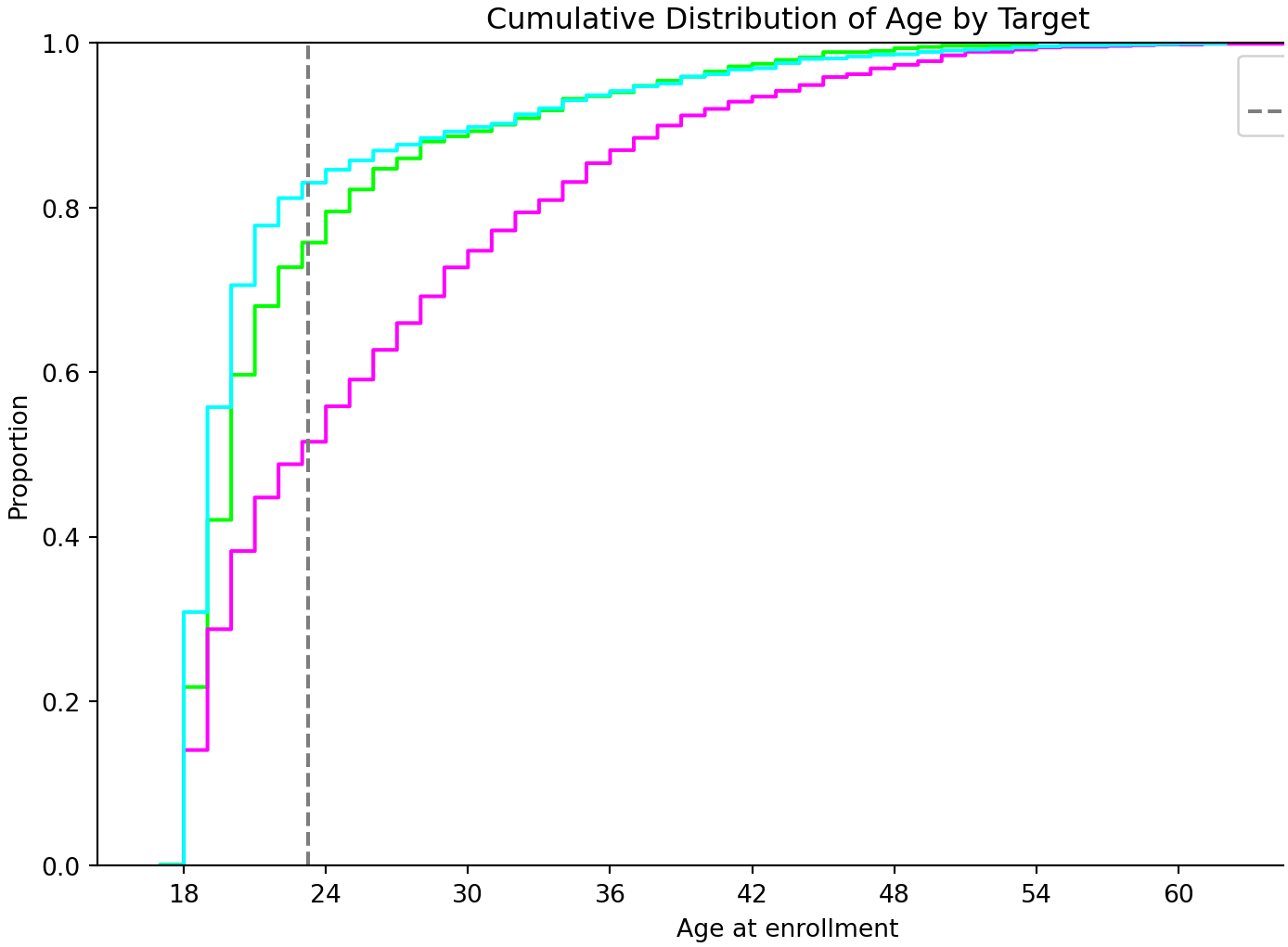
▶ Code



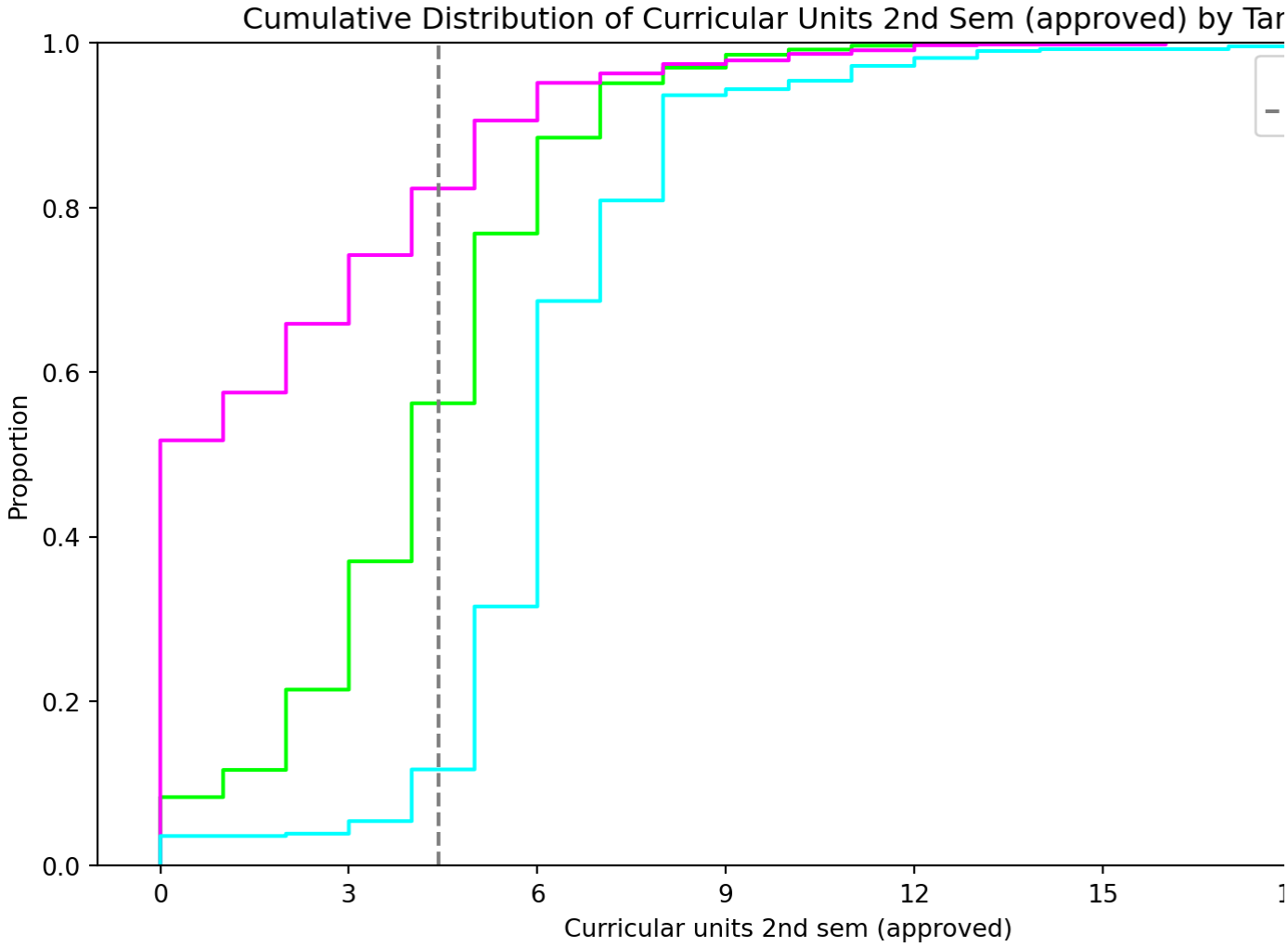
3.0.5 Table 5: Target Distribution by Debtor Status (Percentages)

Debtor	Dropout	Enrolled	Graduate
0	28.19	17.97	53.84
1	64.01	17.74	18.25

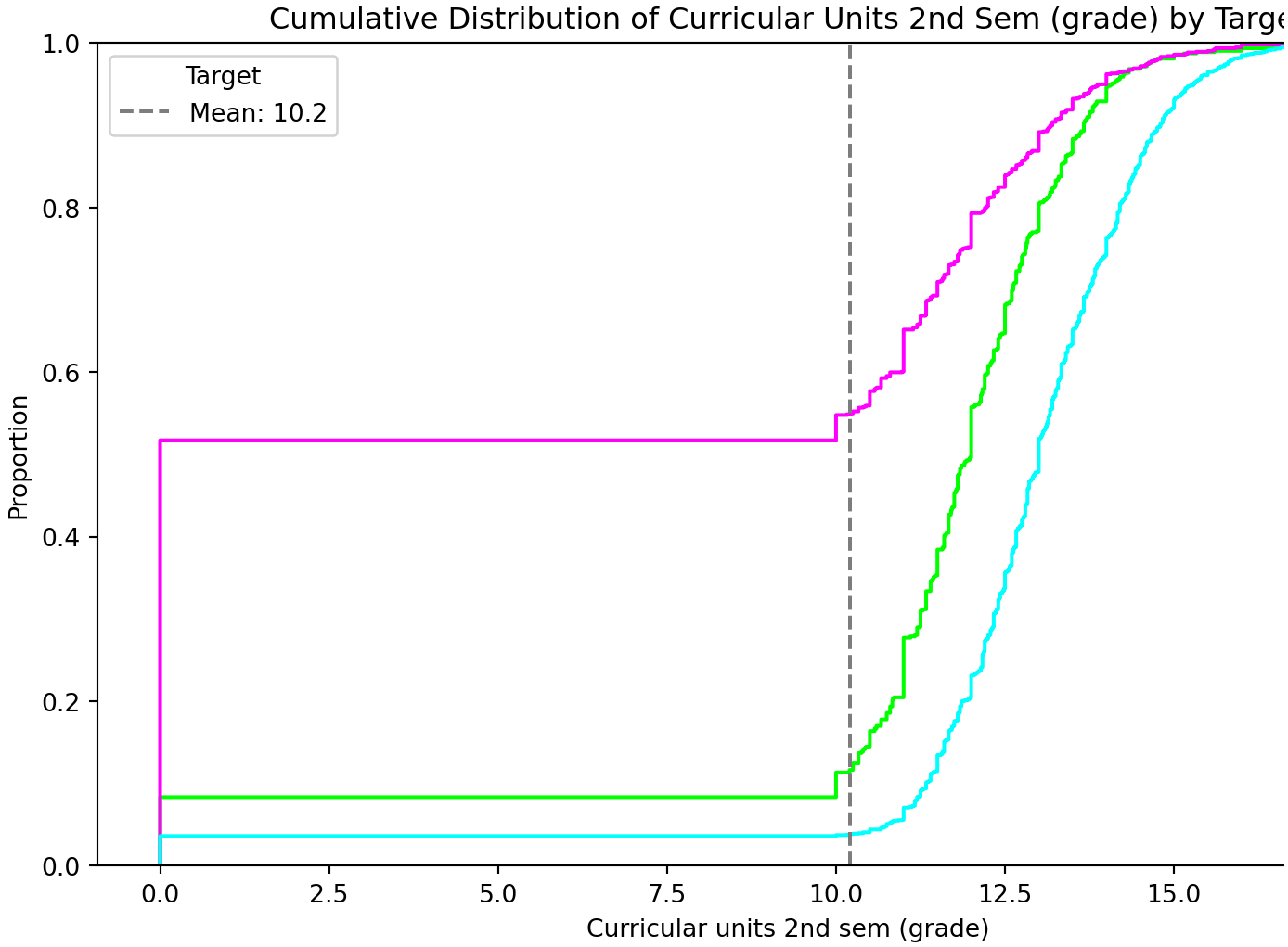
▶ Code


Cumulative Distribution of Age by Target
1.0
0.8
0.6
0.4
0.2
0.0
Proportion
18
24
30
36
42
48
54
60
Age at enrollment

▶ Code


Cumulative Distribution of Curricular Units 2nd Sem (approved) by Tar
1.0
0.8
0.6
0.4
0.2
0.0
Proportion
0
3
6
9
12
15
Curricular units 2nd sem (approved)

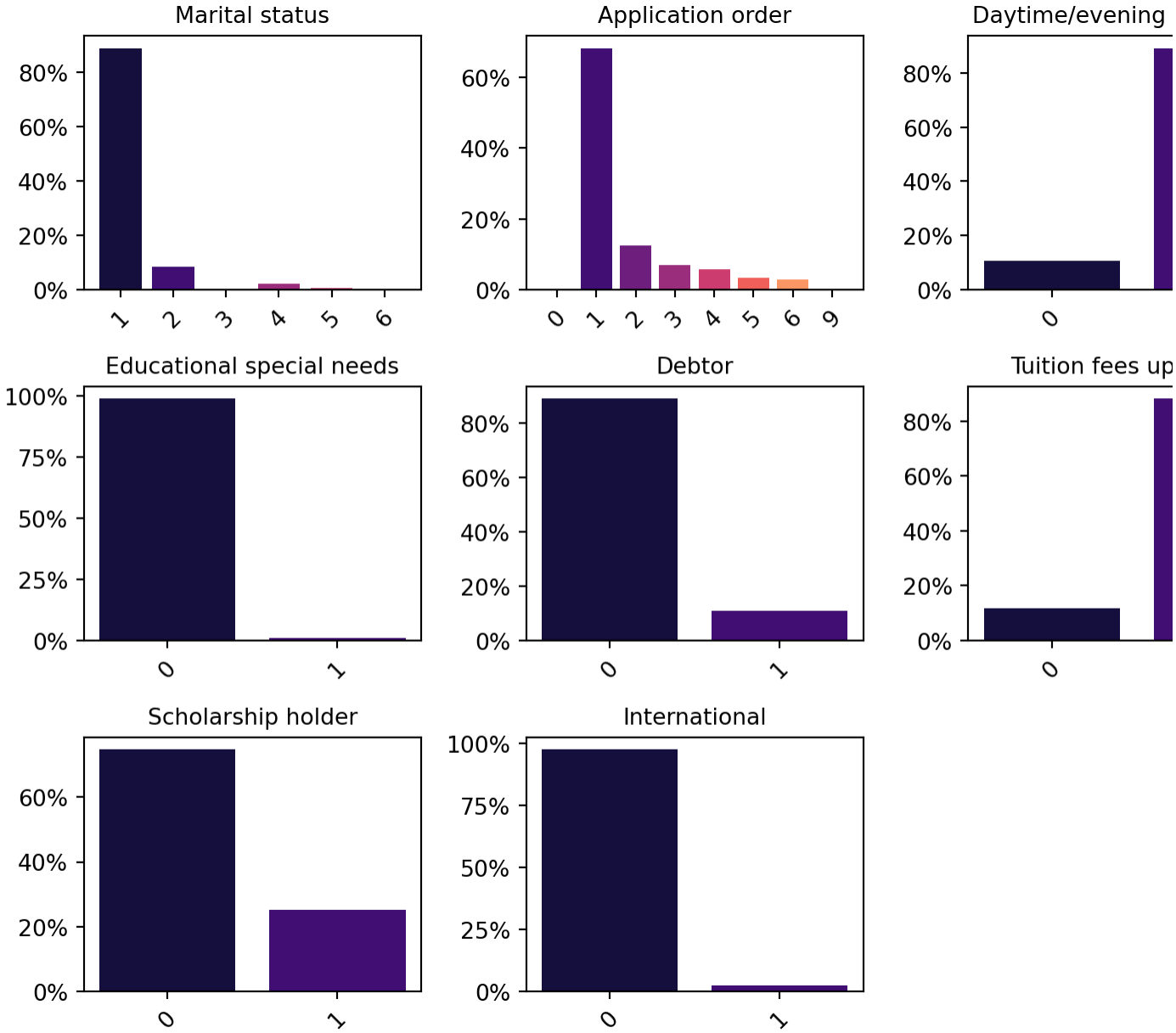
▶ Code



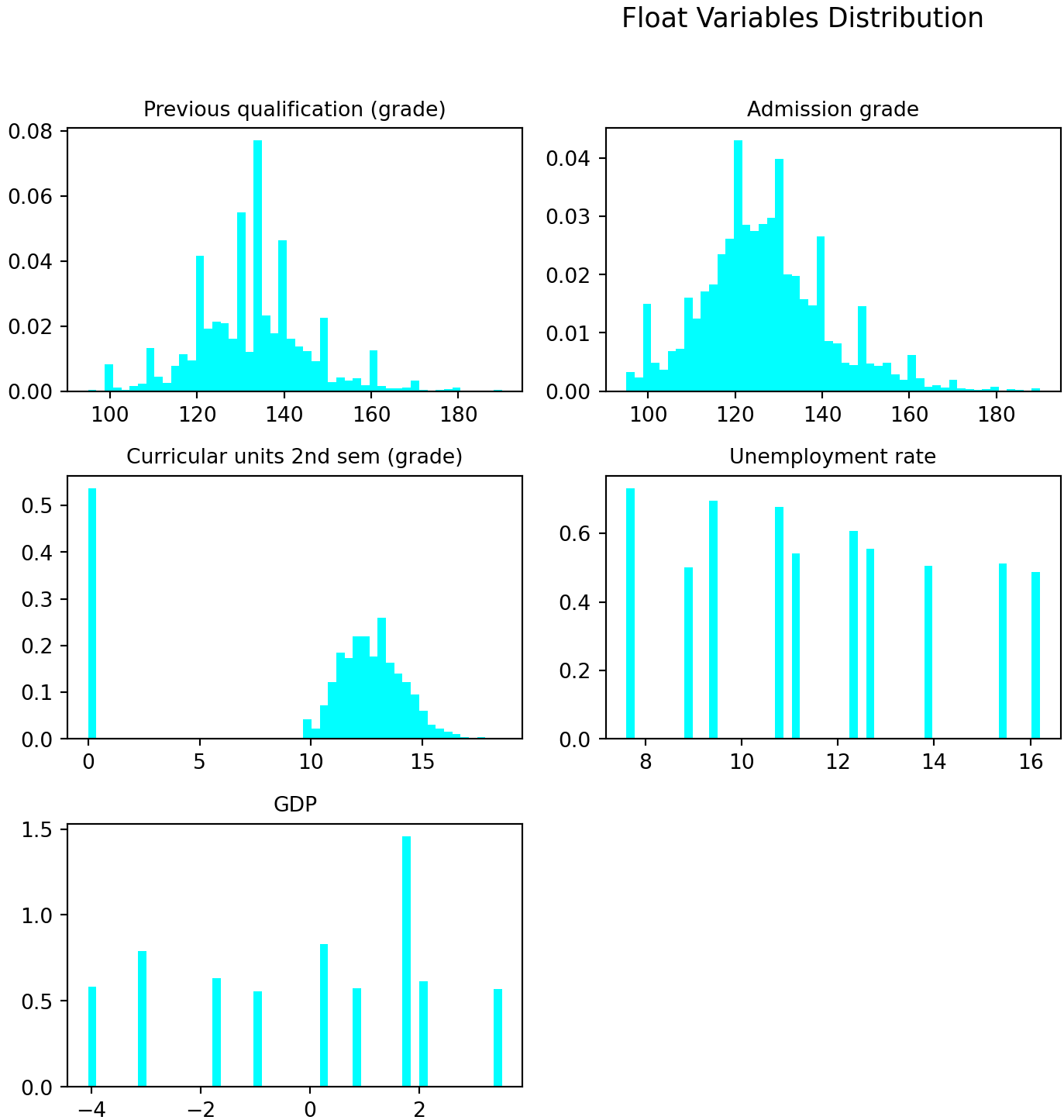
▶ Code

```
Categorical columns:
['Marital status', 'Application order', 'Daytime/evening attendance', 'Displaced',
'Educational special needs', 'Debtor', 'Tuition fees up to date', 'Gender', 'Scholarship
holder', 'International']
```

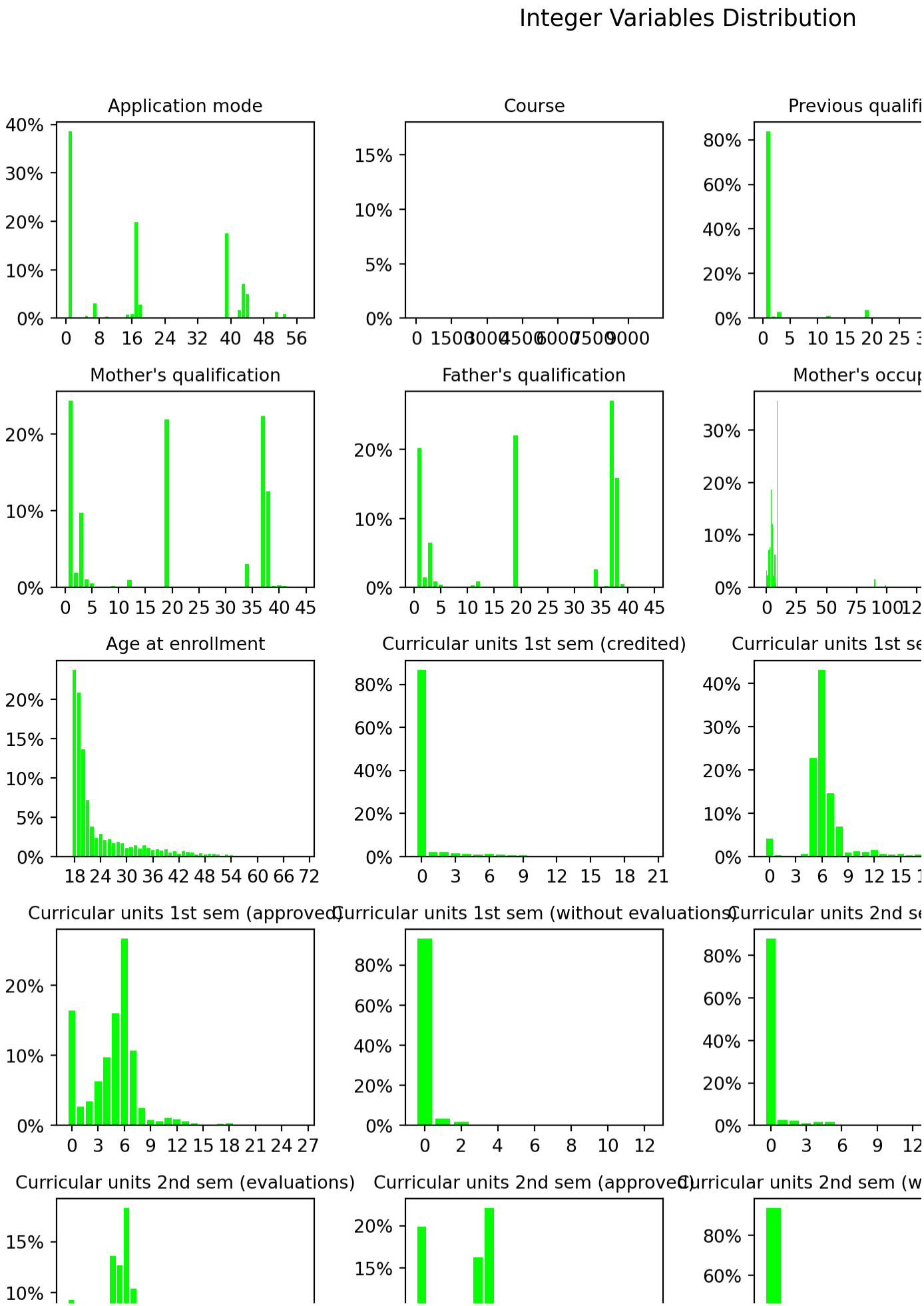
▶ Code

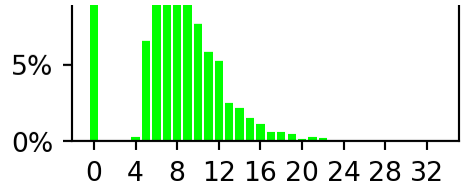

Marital status
80%
60%
40%
20%
0%
1
2
3
4
5
6
Application order
60%
40%
20%
0%
0
1
2
3
4
5
6
9
Daytime/evening
80%
60%
40%
20%
0%
0
Educational special needs
100%
75%
50%
25%
0%
0
1
Debtor
80%
60%
40%
20%
0%
0
1
Tuition fees up
80%
60%
40%
20%
0%
0
Scholarship holder
60%
40%
20%
0%
0
1
International
100%
75%
50%
25%
0%
0
1

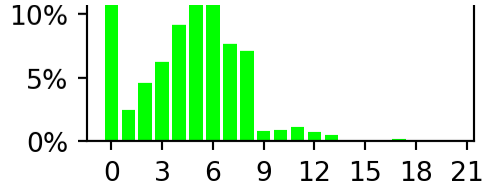
▶ Code

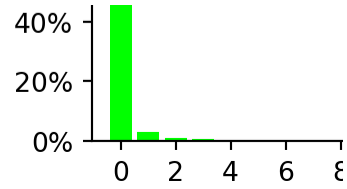

Float Variables Distribution
Previous qualification (grade)
Admission grade
Curricular units 2nd sem (grade)
Unemployment rate
GDP

▶ Code


Integer Variables Distribution
Application mode
Course
Previous qualifi
Mother's qualification
Father's qualification
Mother's occup
Age at enrollment
Curricular units 1st sem (credited)
Curricular units 1st se
Curricular units 1st sem (approved)
Curricular units 1st sem (without evaluations)
Curricular units 2nd se
Curricular units 2nd sem (evaluations)
Curricular units 2nd sem (approved)
Curricular units 2nd sem (w

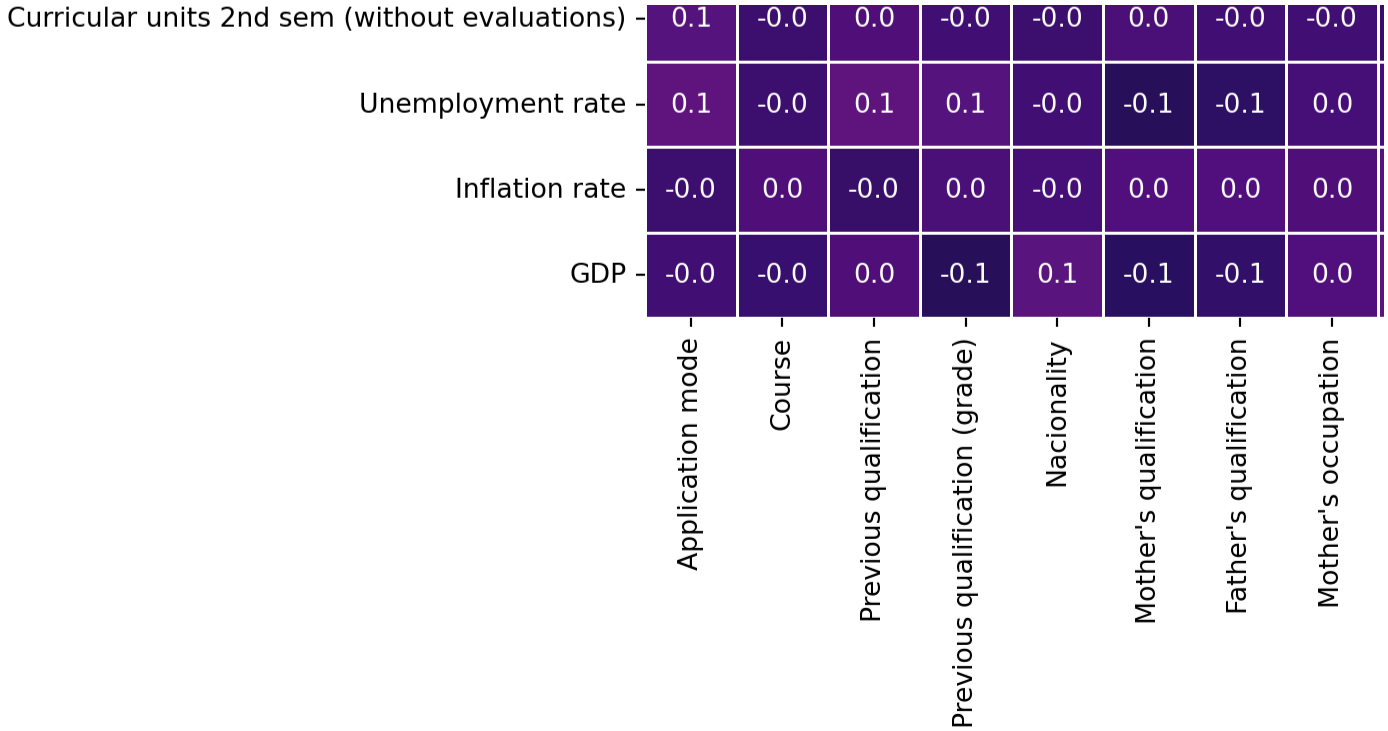






▶ Code

Application mode	1.0	-0.1	0.4	-0.0	0.0	0.1	0.0	0.0
Course	-0.1	1.0	-0.1	-0.0	0.0	0.1	0.0	0.0
Previous qualification	0.4	-0.1	1.0	0.1	-0.0	0.0	0.0	0.0
Previous qualification (grade)	-0.0	-0.0	0.1	1.0	0.0	-0.0	-0.0	-0.0
Nacionality	0.0	0.0	-0.0	0.0	1.0	-0.0	-0.1	0.0
Mother's qualification	0.1	0.1	0.0	-0.0	-0.0	1.0	0.4	0.3
Father's qualification	0.0	0.0	0.0	-0.0	-0.1	0.4	1.0	0.2
Mother's occupation	0.0	0.0	0.0	-0.0	0.0	0.3	0.2	1.0
Father's occupation	-0.0	0.0	0.0	-0.0	0.0	0.2	0.2	0.5
Admission grade	-0.0	-0.1	0.1	0.6	0.0	-0.0	-0.0	-0.0
Age at enrollment	0.5	-0.1	0.4	-0.1	0.0	0.2	0.1	0.1
Curricular units 1st sem (credited)	0.4	-0.2	0.3	0.0	0.0	-0.0	-0.0	-0.0
Curricular units 1st sem (enrolled)	0.1	0.2	0.1	0.0	0.0	0.0	-0.0	0.0
Curricular units 1st sem (evaluations)	0.2	0.0	0.1	-0.1	0.0	0.0	0.0	-0.0
Curricular units 1st sem (approved)	-0.1	0.1	-0.0	0.1	-0.0	-0.0	0.0	-0.0
Curricular units 1st sem (grade)	-0.2	0.1	-0.0	0.2	0.0	-0.0	-0.0	-0.0
Curricular units 1st sem (without evaluations)	0.0	-0.1	0.0	0.0	0.0	-0.0	-0.0	-0.0
Curricular units 2nd sem (credited)	0.3	-0.2	0.3	-0.0	0.0	0.0	-0.0	-0.0
Curricular units 2nd sem (enrolled)	0.0	0.2	0.1	0.0	-0.0	0.0	-0.0	0.0
Curricular units 2nd sem (evaluations)	0.1	0.0	0.1	-0.1	-0.0	0.0	-0.0	-0.0
Curricular units 2nd sem (approved)	-0.1	0.2	-0.1	0.1	-0.0	-0.0	0.0	0.0
Curricular units 2nd sem (grade)	-0.2	0.1	-0.0	0.2	0.0	-0.0	-0.0	-0.0



The (UCI Student Dropout Dataset) does depict significant class imbalance with the Graduate class representing 49.9%, (n = 2,209) of the distribution followed by the Dropout Category class at 32.1%, (n = 1,421) and finally the Enrolled class at 17.9%, (n = 794). I observed several meaningful relationships across the feature groups (i.e., demographic, financial and academic). For example, demographic and financial features displayed strong correlations to the target. When I ran the crosstab on the gender feature, it showed that males were more likely to dropout and females more likely to graduate. Those students who held scholarships also showed higher graduation rates compared to non-scholarship students, and were less likely to drop out. Further, those students who were in financial distress as indicated by being coded as debtors had lower graduation rates than those coded as non-debtors. Concerning academic features, second-semester grades and approved curricular units showed clear dispersion between the target classes, which may indicate importance pertaining to class prediction.

As mentioned the dataset does display significant class imbalance. This class imbalance if left un-checked can lead to model bias towards the majority class (**mduma202?**). I also noticed that many of the categorical features were encoded as integers which may lead to false interpretation when conducting correlation analysis if not addressed Raschka et al. (2022)]. Another limitation that this data presents is its lack of temporal features. As the data represents a snapshot in time, models may not be able to capture the full complexity of predicting student dropout due to the lack of dynamic changes over time. Finally, as this data stems from a single institution, the findings may present limitation concerning generalization. Though I am not as concerned with this as the purpose of this study is to test which model and data balancing techniques when paired has the highest accuracy.In the following

paragraphs I explain the full machine learning pipeline that I employed to test and evaluate the multi-class classifiers as well as the class balancing techniques that accompanied them.

4 Analysis and Results

All model pipeline and experimentation was conducted in Google Colab using a jupyter notebook[link]. The first step was to encode the target into numeric class labels (i.e., 'Dropout': 0, 'Enrolled': 1, and 'Graduate' : 2). Next, I split the dataset into training and validation sets using a 80:20 train_test_split and stratification (Raschka et al., 2022). All numeric and categorical features were preprocessed by imputation and one-hot encoding (Raschka et al., 2022).

After the data was pre-processed I implemented the class imbalancing techniques into the pipeline (i.e., RUS, ROS,SMOTE, SMOTE-ENN and SMOTE-Tomek) on the training set. Each data balancing technique was paired with a multi-class classifiers (i.e., RF, XGBoost, LightGBM, and CatBoost). All models were evaluated with multi-class metrics however F1-score was the primary metric (Mduma, 2023).The effect of all model-balancing combinations were compared, and finally once the best model pipeline was selected I conduced a detailed evaluation using a confusion matrix and permutation feature importance (Mduma, 2023).

Permutation Feature importance is calculated by observing differences in model error (i.e., decreases or increases) when feature values are permutated(Realinho et al., 2022). If premutating feature values causes big differences in the model error, the feature is important to the overall model (see formula below).

- **Permuation Feature Importance Formula**

$$PI_j = E_{\text{perm}(j)} - E_{\text{base}}$$

Where:

- PI_j = the permutation importance score for the feature j
- E_{base} = the base model error (i.e., the performance metric) computed on the original dataset
- $E_{\text{perm}(j)}$ = the model error after random permutation of the values of feature j
- j = the index of the feature being evaluated
- E = the prediction error metric used for evaluation (e.g., mean squared error, log loss or error rate)

To assess the error when conducting permutation I used F1-score as the metric, which according to Realinho et al. (2022), is most adequate when dealing with an imbalanced data set. The F1-score measures the accuracy of classification models assessing how well they predict the positive class by balancing both precision and recall. In the context of selecting "important" features, if permutation causes significant decreases in F1-score, then a feature is deemed important (see formula below).

- **F1-Score Formula**

$$F_1 = \frac{2PR}{P+R}$$

Where:

- F_1 = The $F1$-score which is the harmonic mean of precision and recall
- P = Precision, which is the proportion of predicted positive that are true positives
- R = Recall, which is the ratio of actual positive that are correctly predicted
- 2 = The weighting factor that yields the harmonic mean and it penalizes extreme imbalances between P and R (Realinho et al., 2022).

As mentioned previously in this capstone project Permutation Feature Importance was strictly used as an evaluation criteria on the final machine learning pipeline to assess which features were most important (Realinho et al., 2022). Specifically, permutation of feature importance was conducted on the XGBoost and SMOTE Tomek pipeline which achieved the highest accuracy (i.e., 77.82%). Below I discuss in more detail each of the classifiers and data balancing techniques used in this project.

RF is considered an ensemble learning method that builds upon many decision trees and then aggregates their prediction thus improving model accuracy while reducing overfitting (Ho, 1995).

- **Random Forest Formula**

$$\hat{y} = \frac{1}{B}\sum_{b=1}^{B} T_b(x)$$

Where:

- $\hat{y}$ = The final predicted out of the RF model
- B = The total number of decision trees in the RF
- $T_b(x)$ = The prediction of b-th decision tree for input x
- x = The feature vector (i.e., the feature observation)
- $\sum_{b=1}^{B}$ = The aggregation across all trees

XGBoost operates on the premise of gradient boosting and builds trees in a sequential manner where each new tree corrects the errors of the previous tree (Chen & Guestrin, 2016).

- **Extreme Gradient Boosting Formula**

$$\hat{y}_i = \sum_{k=1}^{K} f_k(x_i), \quad f_k \in \mathcal{F}$$

Where:

- $\hat{y}_i$ = The predicted output for the observation i
- K = The number of trees (i.e., the number of boosting iterations)
- $f_k(x_i)$ = The prediction from the $k-th$ decision tree
- x_i = The feature vector for the observation i

- F = The space of all possible decision trees

LightGBM also operates using a gradient boosting method but uses a histogram-based tree framework while it learns to scale large datasets (Ke et al., 2017).

- **Light Gradient Boosting Machine Formula**

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \eta f_t(x_i)$$

Where:

- $\hat{y}_i^{(t)}$ = The updated observation i at iteration t
- $\hat{y}_i^{(t-1)}$ = The prediction from the previous boosting ineration
- η = The learning rate while controlling for the contribution of each new tree
- $f_t(x_i)$ = The prediction of the tree added at iteration t
- x_i = The feature vector for observation i

CatBoost also uses a gradient boosting method but is specifically designed to handle categorical features (Prokhorenkova et al., 2018). Because of the distinct capability I did analyze CatBoost separately rather than integrate it into the pipeline.

- **Categorical Boosting Formula**

$$\hat{y}_i = \sum_{t=1}^{T} \eta f_t(x_i)$$

Where:

- $\hat{y}_i$ = The predicted output for observation i
- T = The total number of trees (i.e., boosting interations)
- η = The learning rate
- $f_t(x_i)$ = The predcition from the tree at interation t
- x_i = The input vector including the catgorical variables

Data Balancing

To address class imbalance I implemented five data balancing techniques on the dataset; 1. Random Under Sampling (RUS), 2. Random Over Sampling (ROS), 3. Synthetic Minority Over Sampling (SMOTE), 4. Synthetic Minority Over Sampling with Edited Nearest Neighbors (SMOTE ENN), and 5. Sythentc Minority Over Sampling with TOMEK Links (SMOTE TOMEK) (Mduma, 2023).

Concerning RUS, this technique works by randomly removing observations from the majority class until the classes are approximatley equal.

Random Under Sampling Formula

$$\alpha_{us} = \frac{N_m}{N_{rM}}$$

Where:

- α_{us} = The under sampling ratio
- N_m = The number of class samples in the minority
- N_{rM} = The number of majority class samples after under sampling

ROS, is the oposite of RUS, and works by randomly duplicating the minority class until the classes are approximatly equaly.

Random Over Sampling Formula

$$\alpha_{os} = \frac{N_{rm}}{N_m}$$

Where:

- α_{os} = Is the oversampling ration
- N_m = The original number of the minority class samples
- N_{rM} = The number of samples in the minority class after over-sampling

SMOTE works by generating a new synthetic minority class, which is done by interpolating the minority class using the nearest neighbor until both the minority and majority class match.

Synthetic Minority Oversampling Formula

$$x_{\text{new}} = x_i + \lambda \left(x_{nn} - x_i\right)$$

Where:

- x_{new} = The newly generated synthetic minority class sample
- x_i = The original minority class instance
- x_{nn} = Is on of the k-nearest neighbors of x_i
- λ = Is the random interpolation where 0 $\leq \lambda \leq$ 1
- k = The total number of nearest neighbors considered

SMOTE ENN works the same as SMOTE but adds a cleaning step that removes miss-classfied classes due to the nearest neighbor step, this reduces statistical noise and overlap.

Synthetic Minority Oversampling with Edited Nearest Neighbors Formula

$$D^* = \text{ENN}\big(\text{SMOTE}(D)\big)$$

Where:

- D = The original imbalanced dataset
- $SMOTE(D)$ = The dataset after synthetic minority over samples are generated
- $ENN(.)$ = The rule that edits the nearest neighbors that may be miss-classified or ambiguous
- D^* = The final cleaned and balanced dataset

SMOTE TOMEK also works the same as SMOTE but uses TOMEK links, which remove pairs of nearest neighbors instances from the majority class that are in close proximity to eachother.

Synthetic Minority Oversampling with TOMEK Links Formula

$$D^* = D_{\text{SMOTE}} \setminus TL$$

Where:

- D_SMOTE = The dataset after minority synthetic over sampling
- TL = The TOMEK link sets between the minority and majority classes
- $\setminus$ = The removal operator (i.e.,set subtraction)
- D^* = The final balanced dataset after overlapping TOMEK links are removed.

By implementing the above data balancing techniques I ensured that the decision boundary was cleaned thus reducing statistical noise and improving model performance (Mduma, 2023). The five data balancing techniques (Random Over Sampler (ROS), Random Under Sampler (RUS), SMOTE, SMOTE ENN, and SMOTE Tomek, in addition to the original unbalanced dataset, were paired with each of the multi-class classifiers (i.e., Random Forest, XGBoost, and LightGBM) to assess which pair yielded the most optimal results. For example, I assessed performance in six instances: 1. On the original unbalanced dataset, 2. on the ROS balanced dataset, 3. on the RUS balanced dataset, 4. on the SMOTE balanced dataset, 5. on the SMOTE ENN balanced dataset, and 6. on the SMOTE TOMEK balanced dataset.

Concerning model evaluation, specifically for measures of accuracy, I chose metrics that are particularly robust against unbalanced datasets. These included macro-averaged Precision, macro-averaged Recall, and macro-averaged F1-Score ($F1_{\text{macro}}$). The macro-averaged F1-Score, in particular, was the primary criterion used to identify the best performing model, as it treats each class equally. I also considered overall accuracy and the weighted-averaged F1-Score ($F1_{\text{weighted}}$) (Raschka et al., 2022). Macro-averaged metrics provide a balanced view of model performance across all classes, preventing excellent performance on majority classes from masking poor performance on minority classes.

As previously mentioned, this project utilized the F1-Score, a specific instance of the F-measure, primarily its macro-averaged and weighted-averaged forms, rather than a single general F-measure (Raschka et al., 2022). The fundamental formula for an F1-Score is:

$$\text{F1-Score} = \frac{2PR}{P+R}$$

Where:

PP = Precision for a given class RR = Recall for a given class And for a specific class:

$$P = \frac{TP}{TP+FP}$$

$$R = \frac{TP}{TP+FN}$$

Where:

$TPTP$ = True Positive $FNFN$ = False Negative $FPFP$ = False Positive

In this project, the macro-averaged F1-Score was calculated by computing the F1-Score independently for each class and then taking the unweighted mean. The weighted-averaged F1-Score was calculated by computing the F1-Score for each class and then averaging them by the number of true instances for each class (Raschka et al., 2022).

4.1 Modeling and Results

▶ Code

```
Target classes: ['Dropout', 'Enrolled', 'Graduate']
Class mapping: {'Dropout': 0, 'Enrolled': 1, 'Graduate': 2}
```

▶ Code

```
Train shape: (2831, 36) Val shape: (708, 36)
```

▶ Code

```
Categorical cols: 10
Numeric cols: 26
```

▶ Code

▶ Code

▶ Code

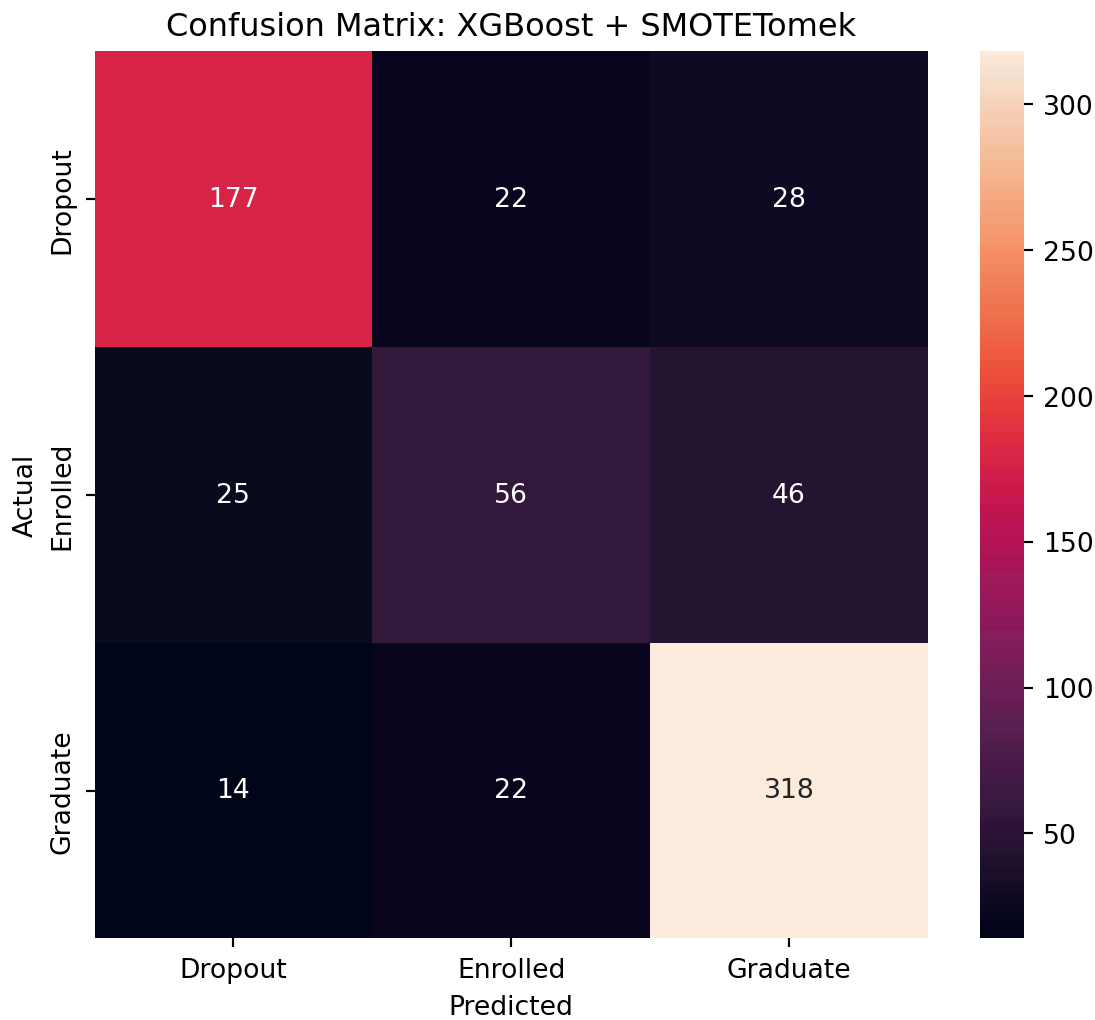
▶ Code

4.1.1 Table 6: Comparative Performance of Models with Different Sampling Techniques (Macro F1-Score Sorted)

Sampler	Model	Accuracy	Precision (macro)	Recall (macro)	F1-Macro	F1-Weighted
SMOTETomek	XGBoost	0.7782	0.7273	0.7052	0.7131	0.7708
SMOTETomek	LightGBM	0.7768	0.7281	0.7037	0.7124	0.7692
SMOTETomek	RandomForest	0.7684	0.7152	0.7037	0.7086	0.7655
SMOTE	XGBoost	0.7768	0.7215	0.7008	0.7079	0.7686
None	XGBoost	0.7811	0.7270	0.6991	0.7070	0.7699

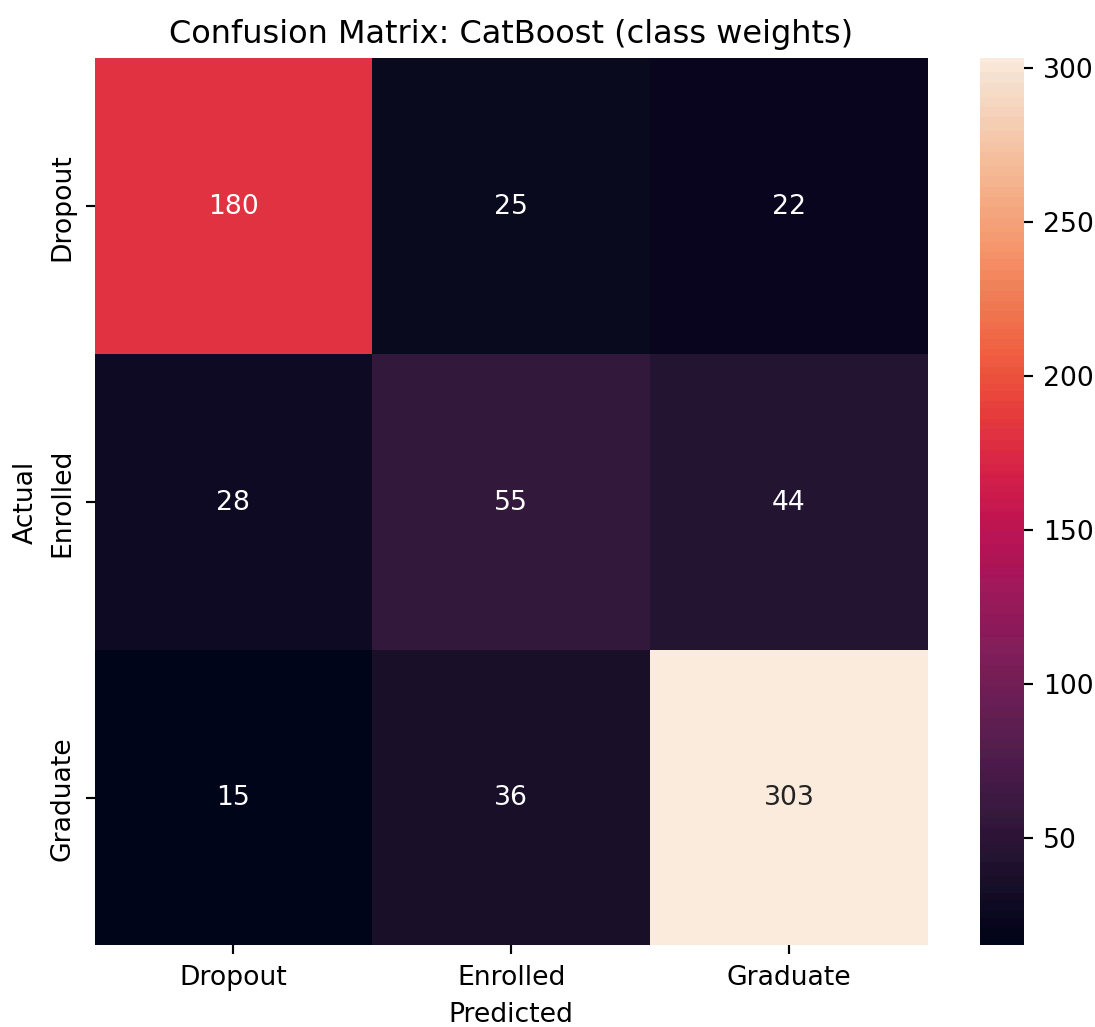
▶ Code

```
Best sampler: SMOTETomek
Best model: XGBoost
accuracy           0.7782
precision_macro    0.7302
recall_macro       0.7063
f1_macro           0.7150
f1_weighted        0.7710
dtype: float64

Classification Report (macro focus):

              precision    recall  f1-score   support

     Dropout       0.82      0.78      0.80       227
    Enrolled       0.56      0.44      0.49       127
    Graduate       0.81      0.90      0.85       354

    accuracy                           0.78       708
   macro avg       0.73      0.71      0.72       708
weighted avg       0.77      0.78      0.77       708
```



▶ Code

```
CatBoost Results:
accuracy           0.7599
precision_macro    0.7008
recall_macro       0.6940
f1_macro           0.6969
f1_weighted        0.7568
dtype: float64

Classification Report (macro focus):

              precision    recall  f1-score   support

     Dropout       0.81      0.79      0.80       227
    Enrolled       0.47      0.43      0.45       127
    Graduate       0.82      0.86      0.84       354

    accuracy                           0.76       708
   macro avg       0.70      0.69      0.70       708
weighted avg       0.75      0.76      0.76       708
```



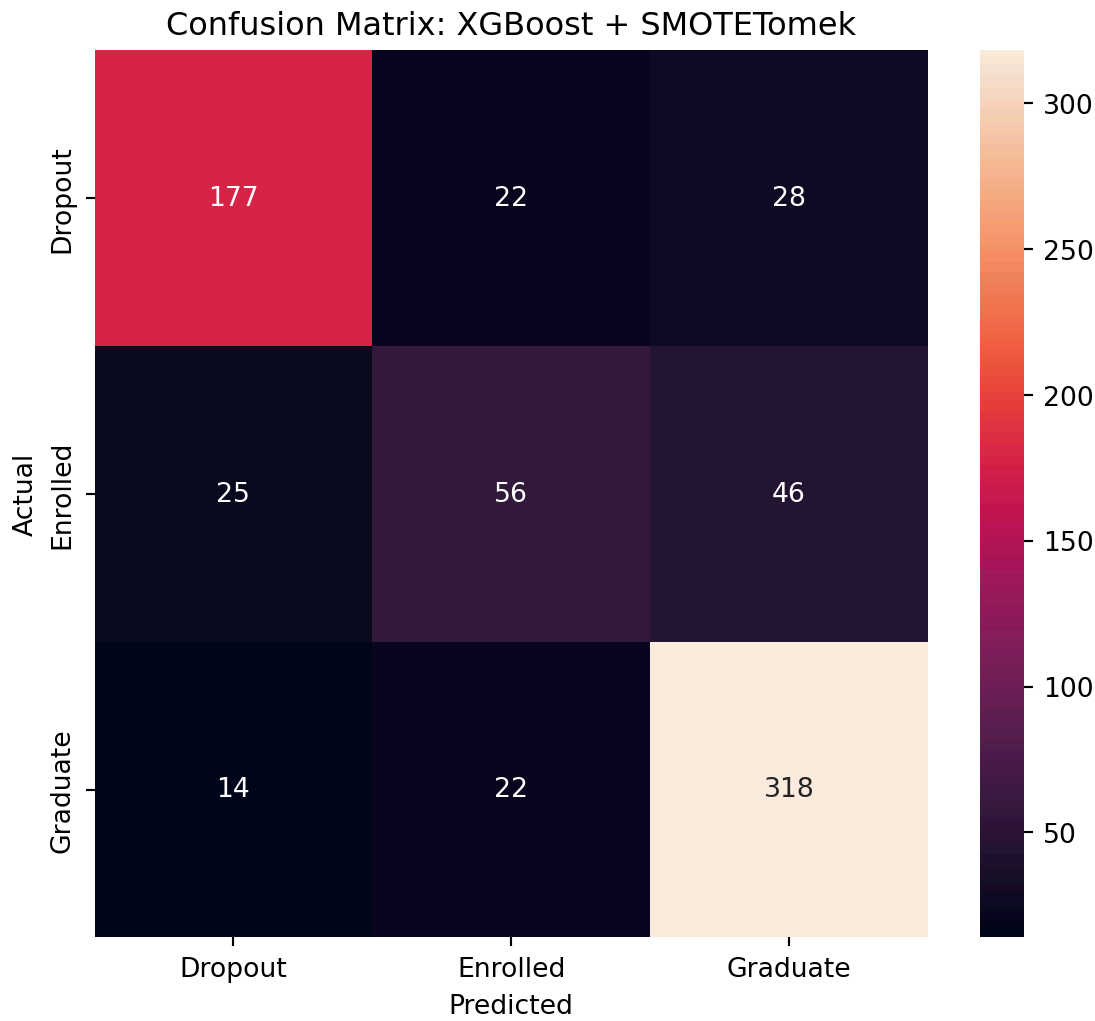
▶ Code

	feature	importance_mean	importance_std
30	Curricular units 2nd sem (approved)	0.187648	0.019643
16	Tuition fees up to date	0.031444	0.011680
24	Curricular units 1st sem (approved)	0.027742	0.006355
3	Course	0.025257	0.010252
10	Mother's occupation	0.021320	0.005959
31	Curricular units 2nd sem (grade)	0.015764	0.005662
19	Age at enrollment	0.014897	0.005074
28	Curricular units 2nd sem (enrolled)	0.014220	0.007198
29	Curricular units 2nd sem (evaluations)	0.010250	0.010510
1	Application mode	0.009378	0.004804
12	Admission grade	0.006683	0.006598
15	Debtor	0.006636	0.004866
22	Curricular units 1st sem (enrolled)	0.005902	0.004234
11	Father's occupation	0.004601	0.006005
18	Scholarship holder	0.004422	0.005459
9	Father's qualification	0.004266	0.007057
35	GDP	0.003711	0.005160
5	Previous qualification	0.003208	0.004559
34	Inflation rate	0.002330	0.003575
23	Curricular units 1st sem (evaluations)	0.002097	0.008426

▶ Code

```
Final Model Performance (Validation Set):
accuracy           0.7782
precision_macro    0.7302
recall_macro       0.7063
f1_macro           0.7150
f1_weighted        0.7710
dtype: float64

Classification Report (macro focus):

              precision    recall  f1-score   support

     Dropout       0.82      0.78      0.80       227
    Enrolled       0.56      0.44      0.49       127
    Graduate       0.81      0.90      0.85       354

    accuracy                           0.78       708
```

```
   macro avg       0.73      0.71      0.72       708
weighted avg       0.77      0.78      0.77       708
```



4.2 Conclusion

The initial exploratory data analysis as well as the results of preliminary modeling provided important insights into the datasets predictive structure and feasibility concerning the use multi-class machine learning approaches. Of all of the features in the dataset, *Curricular units 2nd semester (approved)* emerged as the most informative predictor of the target as indicated by the highest mutual information score. This particular finding indicates that students' academic performance in the second semester is key to determining student outcomes which aligns to prior research concerning the importance of academic progression indicators (e.g., Realinho et al., 2022; Ridwan et al., 2024).

The feature *Curricular units 2nd semester (grade)* showed a generally consistent pattern across both the training and testing datasets as measured by distribution analysis (Raschka et al., 2022).This observation is critical as it shows that the models trained on the training set can also effectively generalize on unseen data (Raschka et al., 2022). Further, this consistency reduces any risk of shift in distribution and supports and validates the cross-validation techniques used in this project (Raschka et al., 2022). The Spearman Rank Correlation Heatmap afforded for a visual representation concerning the

relationship features and the target which helped identify any issues with multicollinearity (Raschka et al., 2022).

Cross-validation was conducted across all of the classifiers used in this project (i.e., RF, XGBoost and LightGBM and CATBoost). Initially, there were issues with datatype. However, the datatype issues were resolved by implementing the appropriate preprocessing for example by label encoding the all categorical features. The adjustments made in the preprocessing step allowed for evaluation consistency across all classifiers and comparable performance metrics (Raschka et al., 2022). The bar plot provided a visual reference for summarizing the mean cross-validation accuracies and gave insights into the model performance.

In conclusion, based on these findings, future research should focus more on feature importance. In particular, future studies should pay attention to *Curricular units 2nd semester (approved)* concerning the model pipeline. Though beyond the scope of this study, a deeper analysis of cross-validation results could be used to select the top performing features. Once those features that are not top predictors are pruned, hyperparameter tuning can be conducted to improve model prediction accuracy (Raschka et al., 2022) . Hyperparameter tuning incorporated with the established pipeline in this study (i.e.,XGBoost and SMOTE Tomek pipeline) could prove to be an effective strategy for accurately predicting student attrition.

---

5 References

Aina, C., Baici, E., Casalone, G., & Pastore, F. (2022). The determinants of university dropout: A review of the socio-economic literature. *Socio-Economic Planning Sciences*, *79*, 101102. https://doi.org/10.1016/j.seps.2021.101102

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. https://doi.org/10.1145/2939672.2939785

Ho, T. K. (1995). Random decision forests. *Proceedings of the Third International Conference on Document Analysis and Recognition*, *1*, 278–282. https://doi.org/10.1109/ICDAR.1995.598994

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, *30*, 3147–3155.

Mduma, N. (2023). Data balancing techniques for predicting student dropout using machine learning. *Data*, *8*(3), 49. https://doi.org/10.3390/data8030049

Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: Unbiased boosting with categorical features. *Advances in Neural Information Processing Systems*, *31*, 6638–6648.

Raschka, S., Liu, Y. H., & Mirjalili, V. (2022). *Machine learning with PyTorch and scikit-learn*. Packt Publishing.

Realinho, V., Machado, J., Baptista, L., & Martins, M. V. (2022). Predicting student dropout and academic success. *Data*, *7*(11), 146. https://doi.org/10.3390/data7110146

Ridwan, A., Priyatno, A. M., & Ningsih, L. (2024). Predict students' dropout and academic success with XGBoost. *Journal of Education and Computer Applications*, *1*(2), 1–8. https://doi.org/10.69693/jeca.v1i2.13